



Addis Ababa Institute of Technology
Center of Information Technology and Scientific
Computing

Department of IT/SW Eng.

**Virtual Blood Bank Mobile Application to Enhance Emergency
Response for Maternal Care in Ethiopia**

Software Design Specification

PREPARED BY: -GROUP-7

Team Members

- 1.Henok Yoseph UGR /7018/16
- 2.Nahom Taye UGR /4073/16
- 3.Abel Tadesse UGR/1059/15
- 4.Abdellah Shafi UGR/2822/16
- 5.Ezra Ayalew UGR/9077/16
- 6.Hailemariam Libassie UGR/3313/16
- 7.Adomiyas Daniel Ugr/7413/15

Date:29 November 2025

Advisors: Aderaw Semma

Table of Contents

List of Tables	3
List of figures	4
Definitions, Acronyms, Abbreviations	5
1. Introduction	8
1.1 Purpose	8
1.2 General Overview	9
1.3 Development Methods & Contingencies	9
2. System design model	12
2.1 Subsystem decomposition	12
2.2 Hardware/software mapping	14
2.3 Access control	Error! Bookmark not defined.
3. Object Model	18
3.1 Class Diagram	18
3.2 Sequence Diagram	18
3.2 State chart Diagram	21
4. Detailed Design	21
References	41

List of Tables

Table Number Table Title

Table 1 Facility Class

Table 2 Attributes Description for Facility Class

Table 3 Operation Description for Facility Class

Table 4 Address Class

Table 5 Attributes Description for Address Class

Table 6 User Class

Table 7 Attributes Description for User Class

Table 8 Operation Description for User Class

Table 9 AdminUser Class

Table 10 Operation Description for AdminUser Class

Table 11 BloodInventory Class

Table 12 Attributes Description for BloodInventory Class

Table 13 Operation Description for BloodInventory Class

Table 14 BloodRequest Class

Table 15 Attributes Description for BloodRequest Class

Table 16 Operation Description for BloodRequest Class

Table 17 Session Class

Table 18 Attributes Description for Class

Table 19 Operation Description for Class

Table 20 Class

Table 21 Attributes Description for Class

Table 22 Operation Description for Class

Table 23 Class

Table 24 Attributes Description for Class

List of figures

Figure Number Figure Title

Figure 1 High-Level System Context Diagram

Figure 2 VBB System Component Diagram

Figure 3 VBB System Deployment Diagram

Figure 4 VBB System Domain Class Diagram

Figure 5 VBB System Detailed Domain Class Diagram

Figure 6 Statechart Diagram for the BloodRequest Object

Figure 7 Sequence Diagram for a Successful Blood Request

Figure 8 Sequence Diagram for a Successful User Login

Definitions, Acronyms, Abbreviations

This section provides the definitions of all terms, acronyms, and abbreviations required to properly interpret this System Design Document.

Definitions

Term	Definition
API (Application Programming Interface)	A set of rules and protocols that allows different software applications to communicate with each other.
Architecture	The fundamental organization of a system, embodied in its components, their relationships to each other and the environment, and the principles governing its design and evolution.
Class	A blueprint for creating objects in an object-oriented programming language, providing initial values for state (attributes) and implementations of behavior (operations or methods).
Client-Server Architecture	A computing model in which a "client" (e.g., the mobile app) requests services from a central "server" (e.g., the backend).
Component	A modular, deployable, and replaceable part of a system that encapsulates implementation and exposes a set of interfaces.
Containerization (e.g., Docker)	A lightweight form of virtualization that packages an application and its dependencies into an isolated unit that can run consistently on any infrastructure.
Domain Model	An object model of the problem domain that represents the real-world concepts and business entities (e.g., Facility, User, BloodRequest).
Invariant	A rule or condition for a class or attribute that must always be true for the system to be in a valid state.

Object-Oriented Design (OOD)	A design paradigm based on the concept of "objects," which can contain data in the form of attributes and code in the form of methods.
Post-condition	A condition that must be true after the execution of an operation.
Pre-condition	A condition that must be true before the execution of an operation.
Statechart Diagram	A diagram that models the dynamic behavior of a single object, showing its lifecycle through various states and the events that cause it to transition between them.

Acronyms and Abbreviations

Acronym Definition

API	Application Programming Interface
AWS	Amazon Web Services
DB	Database
DBaaS	Database as a Service
EBTBS	Ethiopian Blood and Tissue Bank Service
ERD	Entity-Relationship Diagram
FCM	Firebase Cloud Messaging (Google's Push Notification Service)
GCP	Google Cloud Platform
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation

MVT	Model-View-Template (A Django architectural pattern)
OOD	Object-Oriented Design
ORM	Object-Relational Mapper
OS	Operating System
PNS	Push Notification Service
QA	Quality Assurance
REST	Representational State Transfer (An API architectural style)
SDD	System Design Document
SMS	Short Message Service
SRS	Software Requirements Specification
UI	User Interface
UML	Unified Modeling Language
UUID	Universally Unique Identifier
UX	User Experience
VBB	Virtual Blood Bank
VM	Virtual Machine

1. Introduction

The Software Design Specification (SDS) shows the architectural, structural, and technical design of Virtual Blood Bank Mobile Application, which is a system designed to reinforce emergency response to maternal care in the rural areas of Ethiopia. This document derives upon the requirements specified in the Software Requirements Specification (SRS), and is a translation of requirements into a detailed design blueprint, which may be directly translated by engineers to concrete implementation.

Virtual Blood Bank is an application that will be a digital coordination platform operating at the district level, allowing a real-time view of the blood stock in partnering healthcare centers, facilitating inter-facility blood orders, speeding up and ensuring the reliability of emergency blood access in the event of postpartum hemorrhage (PPH). Due to the fact that the application will be utilized by doctors, nurses, hospital administrators and blood bank technicians who will be working in resource constrained settings, the system design will focus on reliability, performance, ease and low bandwidth or intermittent connectivity.

This SDS provides a system architecture, module breakdown, interface, detail about the data structures, workflow and major design decisions. It makes sure that the development team has a clear, explicit and technically detailed knowledge of how the system will be assembled so as to fulfill the functional as well as non-functional requirements as stipulated in the SRS. It further assists the stakeholders to confirm that the final system design is based on the reality of the rural healthcare facilities, limits of the local infrastructure and the dire conditions of the maternal emergencies.

1.1 Purpose

The purpose of this System Design Document (SDD) is to translate the business requirements and business processes for the **Virtual Blood Bank (VBB) mobile application** into a technical design that will be used to develop and implement the application. This document is the primary technical blueprint, building upon the functional and non-functional requirements previously defined in the Software Requirements Specification (SRS).

It serves as the primary reference for software developers, testers, and system architects to ensure the final product meets the functional and non-functional requirements for reducing maternal mortality through efficient blood inventory management.

Specifically, this SDD will detail:

- The overall System Architecture, including the client-server model and component interactions.
- The chosen Technology Stack for both the mobile application and the backend services.
- The detailed Database Design, including the physical schema, data types, and relationships.
- The high-level User Interface (UI) and User Experience (UX) Design, providing a visual framework for the application's layout and flow.

- The Security Architecture, outlining the implementation of access controls, data encryption, and other security measures.
- The Deployment Strategy, describing how the system will be hosted and maintained.

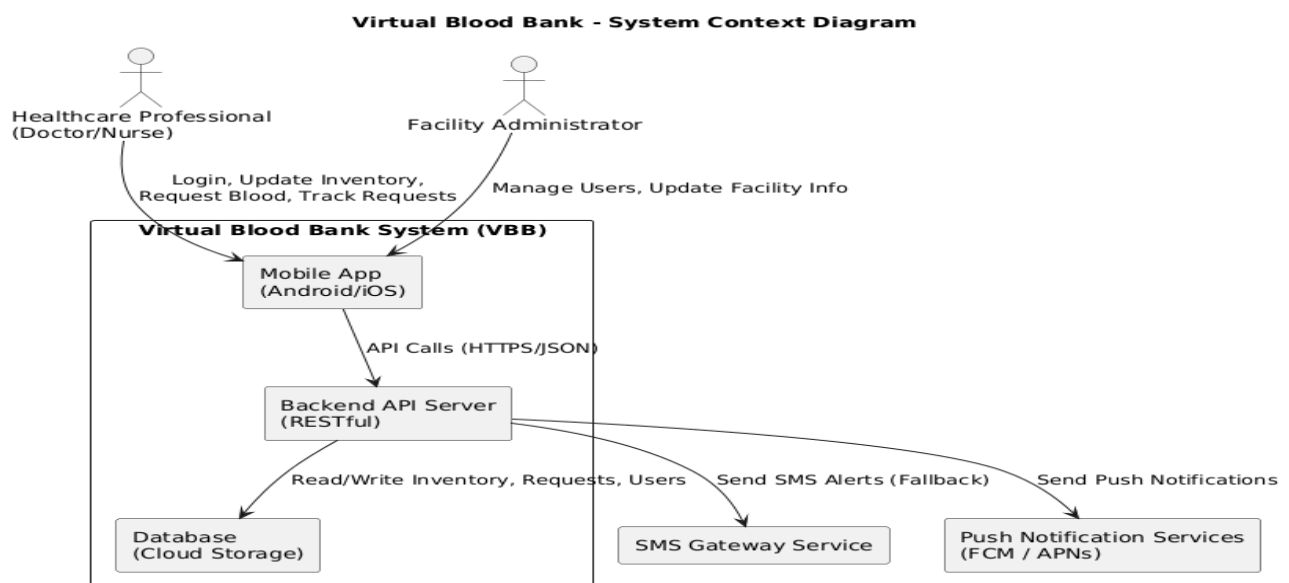
The primary audience for this SDD includes software developers (mobile and backend), system architects, the technical lead. Ultimately, this document will serve as the comprehensive guide for the development team, ensuring that the final product is built in a robust, scalable, and maintainable manner that precisely meets the life-saving objectives of the VBB project.

1.2 General Overview

The Virtual Blood Bank (VBB) is an mHealth system designed to reduce maternal mortality in rural Ethiopia by addressing critical delays in the blood supply chain. The system will function as a real-time, district-level information hub, enabling healthcare professionals to view blood availability in nearby facilities, make urgent requests, and track those requests through to fulfillment. The basic design approach is a modern, mobile-first, client-server architecture. The system will be composed of a cross-platform mobile application for end-users and a centralized, cloud-hosted backend for data management and business logic.

The system architecture is organized into three primary components: a Mobile Client Application, a Backend API Server, and a Cloud Database. The mobile client will be the sole interface for the users (Healthcare Professionals). It will communicate securely over HTTPS with the backend API, which will in turn manage all data persistence in the database and interact with external services for notifications. The primary design goals for this system are Reliability, Security, Scalability, Performance in low-bandwidth environments, Maintainability, and Cost-Effectiveness.

The high-level context diagram below illustrates the system's boundaries, its major components, and its interactions with external actors and systems. This diagram is the definitive high-level design and is consistent with the requirements outlined in the SRS; no updates have been made.



1.3 Development Methods & Contingencies

Development Methods

The design and development of the Virtual Blood Bank (VBB) system will follow a hybrid approach, combining the structured, user-centered principles from the requirements phase with an Agile development methodology for implementation. The technical design of the system is fundamentally Object-Oriented (OO), follows a technology-specific architectural pattern, and will be documented using the Unified Modeling Language (UML) where appropriate.

1. Primary Methodology: Agile (Scrum)

- ✓ **Description:** The project will adopt the Agile framework, specifically a Scrum-like approach. Development will be organized into short, time-boxed iterations called "sprints" (e.g., two weeks), with a focus on delivering functional increments of the software at the end of each cycle. This formal method is described in "The Scrum Guide" by Ken Schwaber and Jeff Sutherland.
- ✓ **Rationale:** An Agile approach was chosen for its flexibility and iterative nature, which is essential for mHealth projects in dynamic environments. It allows for continuous feedback from stakeholders and the ability to adapt to technical or environmental challenges discovered during development, ensuring the final product is closely aligned with real-world user needs.

2. Design Paradigm: Object-Oriented Design (OOD)

- ✓ **Description:** The system's software components will be designed using object-oriented principles. Real-world entities such as Facility, User, BloodInventory, and BloodRequest will be modeled as software objects with distinct attributes and methods.
- ✓ **Rationale:** OOD promotes a modular, reusable, and maintainable codebase. This paradigm makes the system easier to understand, extend, and debug, which is critical for long-term project sustainability.

3. Technology-Specific Approach: Django MVT and Cross-Platform Mobile Framework

- **Description:** The system's design is specifically centered around two key technology choices that define its architecture.
 - ✓ **Backend:** The backend will be built using the Python Django framework, adhering to its Model-View-Template (MVT) architectural pattern.
 - ✓ **Frontend:** The mobile application will be built using a cross-platform framework (e.g., React Native).
- ✓ **Rationale:** The Django MVT pattern was chosen for its "batteries-included" philosophy, providing robust security, an Object-Relational Mapper (ORM), and an admin interface out-of-the-box, which accelerates development. The cross-platform mobile approach was mandated by the need for cost-effectiveness, allowing a single codebase to be deployed on both Android and potentially iOS devices, thereby reducing development and maintenance effort.

4. Documentation Standard: Unified Modeling Language (UML)

- ✓ **Description:** UML, a formal industry standard, will be used to visually model and document the system's architecture and design. This includes Use Case diagrams, System Context diagrams, and Class and Sequence diagrams in later sections.
- ✓ **Rationale:** UML provides a standardized and unambiguous visual language for communicating complex design concepts to all members of the development and testing teams.

5. Alternative Methods Considered

- **Waterfall Methodology:** A traditional Waterfall model was considered but rejected. Its rigid, sequential nature ("design everything, then build everything") lacks the flexibility required for this project. The inability to easily adapt to feedback or changing environmental conditions (like network performance) was deemed too high a risk.
- **Native Mobile Development:** An approach using separate native codebases (Kotlin for Android, Swift for iOS) was also evaluated. This was rejected primarily on the grounds of cost and time. It would require specialized developers for each platform and nearly double the development and maintenance effort, conflicting with the project's cost-effectiveness goals.

6. Design and Development Contingencies

The following contingencies have been identified as potential risks that could alter the design or development direction. Workarounds and alternative plans are outlined for each.

➤ **Contingency 1: Unavailability of a Reliable and Cost-Effective SMS Gateway**

Risk: The design includes SMS notifications as a fallback. If a suitable SMS gateway service with coverage in rural Ethiopia cannot be secured, this fallback feature will be impacted.

Workaround: If an SMS gateway proves unfeasible, the system will rely solely on **in-app push notifications**. The design will be modified to enhance the visibility and persistence of these notifications within the app to ensure they are not missed. Training materials would emphasize the need for users to have data connectivity for critical alerts.

➤ **Contingency 2: Unforeseen Regulatory Hurdles or Data Residency Requirements**

- ✓ **Risk:** Regulations on digital health may impose unforeseen constraints, such as a strict requirement for all data to be hosted on servers physically located within Ethiopia.
- ✓ **Mitigation/Workaround:** The backend architecture will be built on **containerization technologies (Docker)**. This design provides significant portability, making it feasible to migrate the application to a compliant local cloud provider or on-premise infrastructure with minimal refactoring if required by law.

2. System Architecture

This section describes the overall architecture of the Virtual Blood Bank (VBB) system, detailing how it is broken down into logical subsystems and components. The architecture is designed to be modular, scalable, and maintainable, adhering to the design goals outlined in the introduction.

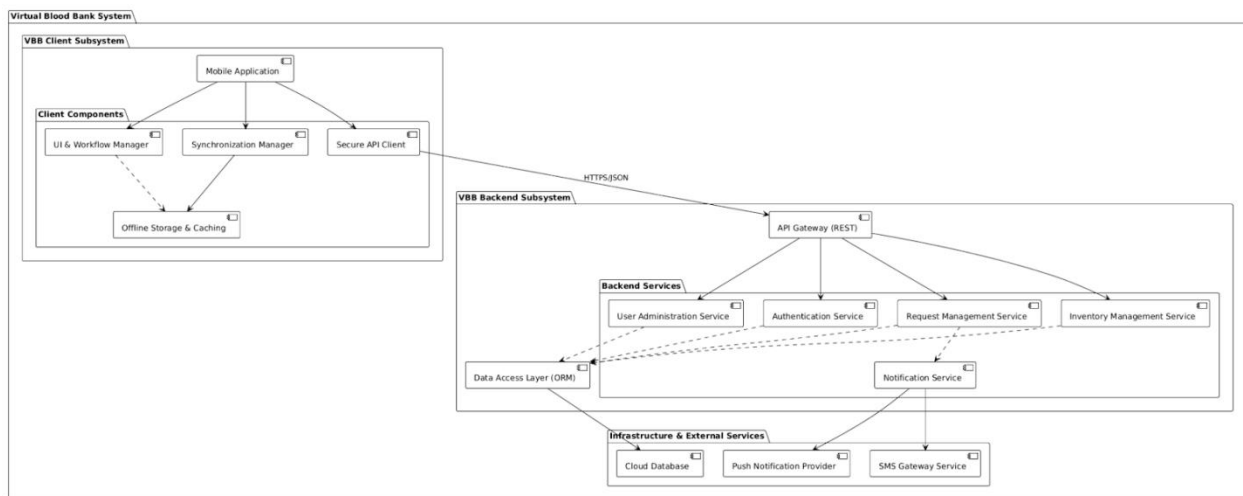
2.1 Subsystem decomposition

The Virtual Blood Bank (VBB) system is structured using a **component-based, service-oriented architecture**. The system is decomposed into three major, high-level subsystems that logically separate the system's core concerns:

- ✓ **VBB Client Subsystem:** Encompasses the mobile application that users interact with directly.
- ✓ **VBB Backend Subsystem:** Contains all the server-side logic, data processing, and core functionalities.
- ✓ **Infrastructure & External Services Subsystem:** Includes the foundational database technology and all third-party services the system depends on.

Each of these subsystems is further decomposed into more granular components, providing a clear, three-level hierarchical view of the system's design. This structure ensures a clean separation of concerns, which enhances scalability and maintainability.

Figure 2: VBB System Component Diagram



Description of Subsystems and Components

1. Level 1: The Virtual Blood Bank System

This represents the entire application boundary, encompassing all software we will build and the immediate infrastructure it relies on.

2. Level 2: Major Subsystems

- ✓ **VBB Client Subsystem:** This is the cross-platform mobile application installed on the Healthcare Professional's device. Its primary responsibility is to provide the user interface and to function reliably even in low-connectivity environments.
- ✓ **VBB Backend Subsystem:** This is the server-side core of the application, hosted in the cloud. It is responsible for executing all business logic, ensuring data integrity, managing user access, and communicating with external services. It is designed as a set of logical services to maintain organization.
- ✓ **Infrastructure & External Services Subsystem:** This subsystem contains all the foundational and third-party components that the VBB system is built upon but does not manage internally.

3. Level 3: Key Components

➤ Inside the Client Subsystem:

- ✓ **UI & Workflow Manager:** Renders all screens and controls the user's journey through the application.
- ✓ **Offline Storage & Caching:** A local database on the device responsible for storing data (like the last known inventory) to ensure the app is usable when offline.
- ✓ **Synchronization Manager:** Manages the queue of outgoing data and intelligently syncs it with the backend when a connection is available.
- ✓ **Secure API Client:** A dedicated component for securely formatting and sending all requests to the Backend API.

➤ Inside the Backend Subsystem:

- ✓ **API Gateway (REST):** The single, secure entry point for all requests from the mobile client. It routes requests to the appropriate internal service.
- ✓ **Authentication Service:** Manages user login, password verification, and session token generation, enforcing security.
- ✓ **Inventory Management Service:** Contains all business logic related to viewing and updating blood stock levels.
- ✓ **Request Management Service:** Handles the creation, tracking, and status updates for all inter-facility blood requests.
- ✓ **User Administration Service:** Manages the creation and deactivation of user accounts by administrators.
- ✓ **Notification Service:** A centralized service responsible for constructing and sending alerts via push notifications and SMS.

- ✓ **Data Access Layer (ORM):** An abstraction layer that provides a consistent and secure way for all services to interact with the Cloud Database.
- **Inside the Infrastructure Subsystem:**
 - ✓ **Cloud Database:** The central, persistent storage for all application data.
 - ✓ **Push Notification Provider (FCM/APNs):** External Google/Apple services used to deliver push notifications.
 - ✓ **SMS Gateway Service:** A third-party service for delivering SMS alerts.

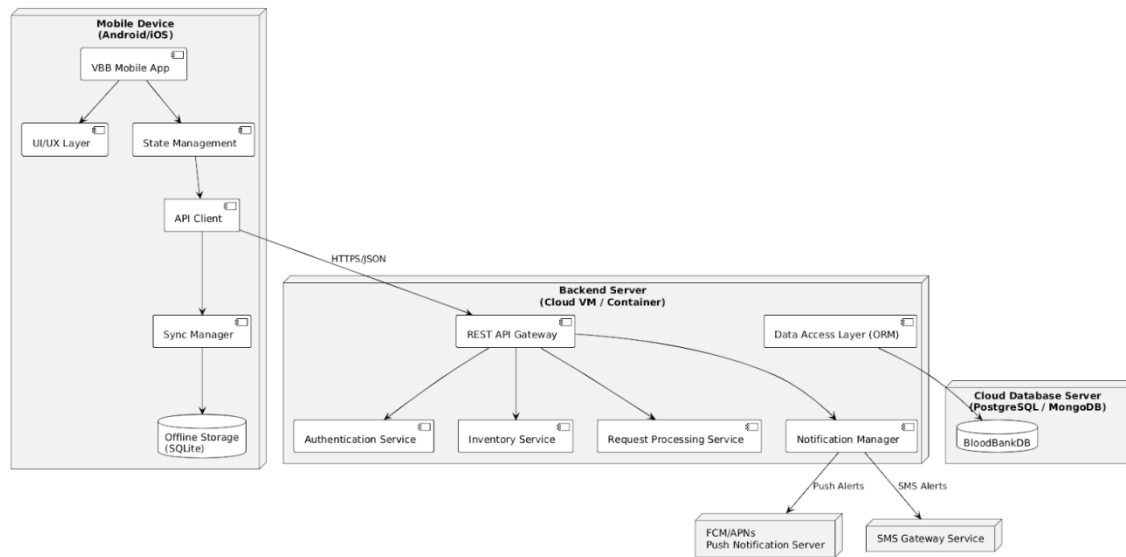
2.2 Hardware/software mapping

This section describes the physical and virtual hardware environment for the Virtual Blood Bank (VBB) system and maps the software components defined in Section 2.1 to this infrastructure. A UML Deployment Diagram is used to visually represent this mapping.

- **Hardware/Node Overview**
 - ✓ **Mobile Device (End-User):** Standard Android or iOS smartphones that will run the VBB mobile application.
 - ✓ **Cloud Application Server:** A cloud-hosted virtual machine or container (e.g., on AWS or GCP) that hosts the backend application logic.
 - ✓ **Cloud Database Server:** A managed database service (DBaaS) that provides persistent storage for all application data.
 - ✓ **External Service Nodes:** Third-party servers that provide essential services like push notifications and SMS.
- **Software Component Mapping**
 - ✓ **On Mobile Device:** The complete VBB Client Subsystem, including the UI, offline storage, and synchronization logic.
 - ✓ **On Application Server:** The VBB Backend Subsystem, including the API Gateway and all logical business services (Authentication, Inventory, etc.).
 - ✓ **On Database Server:** The VBB's central database.
 - ✓ **On External Nodes:** Third-party APIs for Push Notifications and SMS.

Figure 3: VBB System Deployment Diagram

Virtual Blood Bank System - Deployment Diagram



2.3 Access Control

Access control is the mechanism that determines which users can view which data and perform which actions within the system. The VBB system will implement a **Role-Based Access Control (RBAC)** model. This model ensures that users are only granted permissions sufficient to perform their assigned duties, adhering to the principle of least privilege.

2.3.1 Role Definitions

The system will define a set of roles, with each user account being assigned exactly one role. Each role has a specific set of permissions.

Healthcare Professional (Role: PROFESSIONAL)

Description: This is the standard user of the system, including doctors and nurses. Their permissions are focused on the core clinical and operational workflows of managing and requesting blood.

Assigned to: Doctors, Nurses.

Facility Administrator (Role: ADMIN)

Description: This is a privileged user role, typically assigned to one or two trusted individuals per facility or district. Their permissions are focused on user management and system configuration for their designated scope. They do not have permissions for clinical workflows like creating requests.

Assigned to: Lead clinicians, IT support staff, or facility managers.

2.3.2 Permission Matrix

The following table defines the specific actions that each role is permitted to perform. The backend API will enforce these rules for every request.

Action / Feature	Healthcare Professional (PROFESSIONAL)	Facility Administrator (ADMIN)
Authentication		
Log into the System	Yes	Yes
Change Own Password	Yes	Yes
Inventory Management		
View Own Facility Inventory	Yes	Read-Only
Update Own Facility Inventory	Yes	No
View District Inventory	Yes	Yes
Blood Request Management		
Create a Blood Request	Yes	No
View Outgoing Requests	Yes	Read-Only
Respond to Incoming Requests (Accept/Reject)	Yes	No
Update Request Status (e.g., In Transit)	Yes	No

User Management

View Own Profile	Yes	Yes
Create New User Accounts	No	Yes
Deactivate/Reactivate User Accounts	No	Yes
Assign/Change User Roles	No	Yes

General

View Donation Centers	Yes	Yes
-----------------------	-----	-----

2.3.3 Implementation Strategy

Access control will be enforced at two primary layers: the Backend API (for security) and the Mobile Application (for usability).

➤ Authentication and Session Tokens:

- ✓ Upon successful login, the **Authentication Service** will generate a secure **JSON Web Token (JWT)**.
- ✓ This JWT will be sent to the mobile client and will be included in the header of every subsequent API request.
- ✓ Crucially, the JWT payload will contain the userId and the user's role (e.g., "role": "PROFESSIONAL"). The token will be digitally signed by the server to prevent tampering.

➤ Backend API Enforcement (Primary Control):

- ✓ The **API Gateway** is the gatekeeper. For every incoming request, it will first validate the JWT.
- ✓ It will then inspect the role claim within the validated token.
- ✓ Based on the Permission Matrix, the API will either allow the request to proceed to the relevant business service or immediately reject it with a **403 Forbidden** HTTP status code.

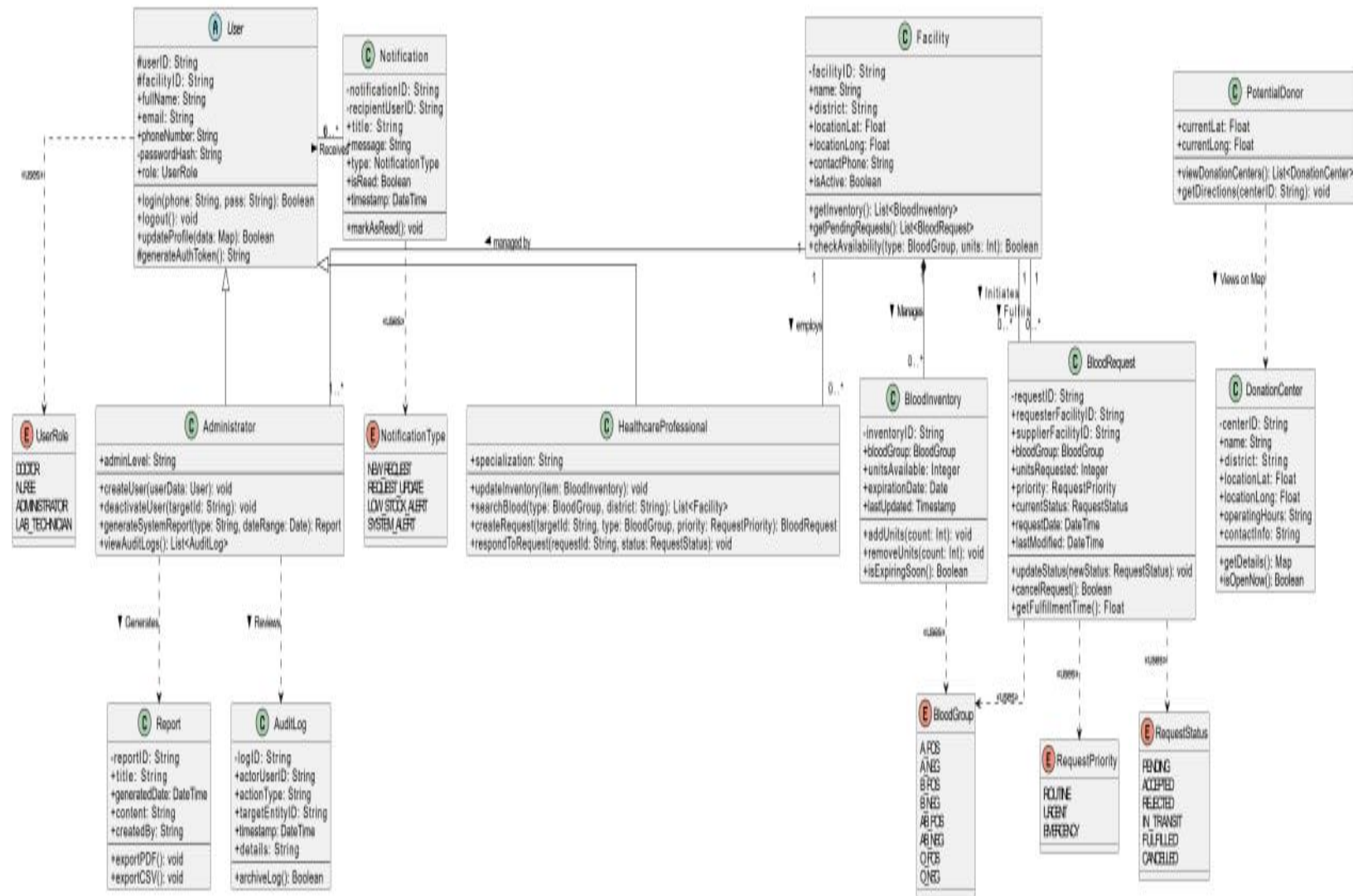
- ✓ This server-side enforcement is the most critical part of the security model and ensures that even if the client-side UI is bypassed, unauthorized actions cannot be performed.
- **Mobile Application UI Adaptation (Usability):**
 - ✓ After logging in, the mobile application will decode the JWT (or receive user profile information) to determine the user's role.
 - ✓ The UI will dynamically adapt based on this role. For example:
 - ✓ A user with the ADMIN role will see a "Manage Users" button on their dashboard.
 - ✓ A user with the PROFESSIONAL role will *not* see this button.
 - ✓ This client-side adaptation improves the user experience by not showing users options they are not permitted to use. However, it is **not** a security measure, merely a convenience.

3. Object Model

This section describes the static structure of the Virtual Blood Bank system using an object-oriented model. It details the classes that make up the system, their responsibilities, and the relationships between them, serving as a blueprint for the source code.

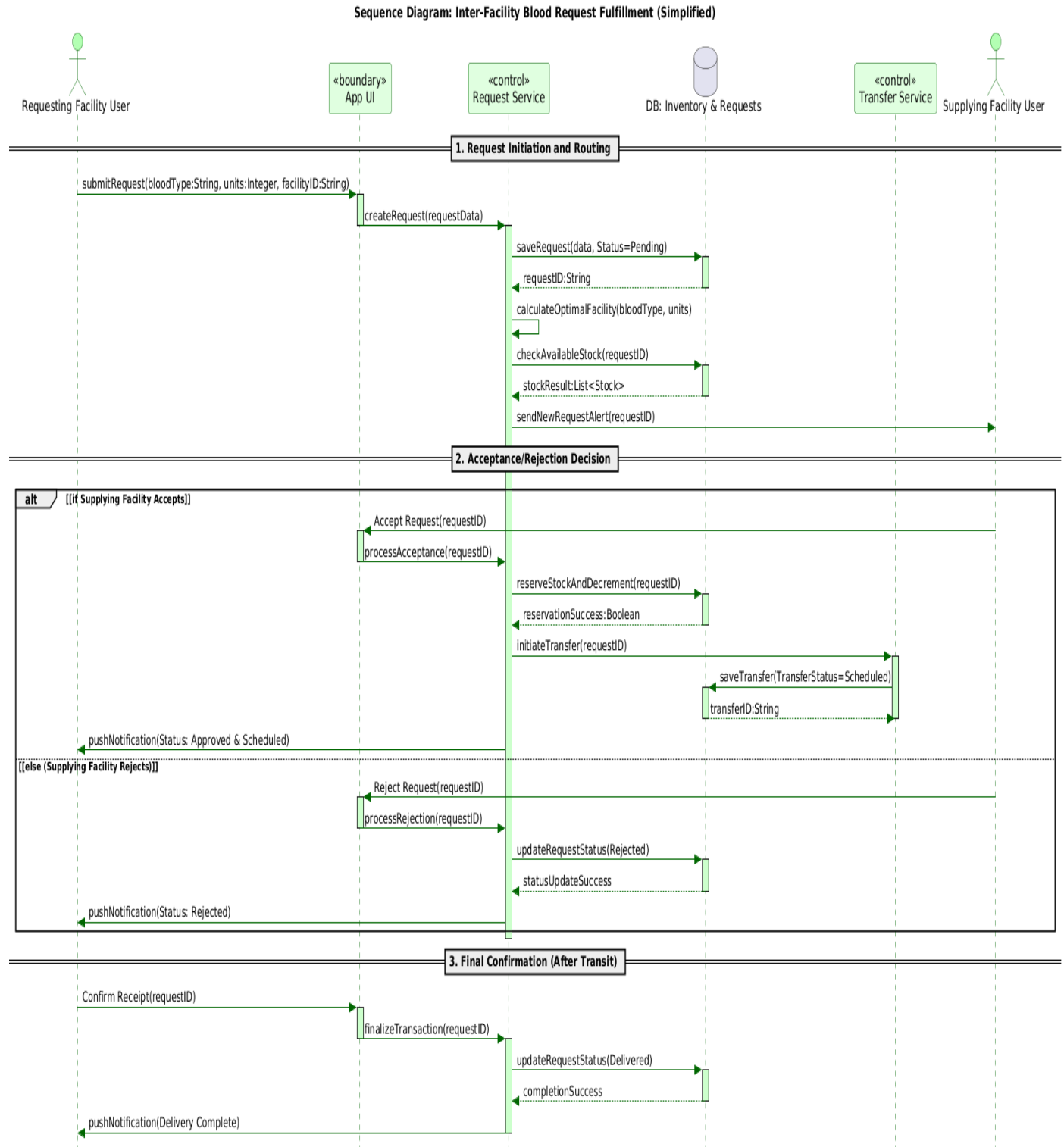
3.1 Class Diagram

The following UML Class Diagram describes the logical structure of the VBB system. It focuses on the core domain classes that represent the primary entities managed by the application. The diagram shows each class's essential attributes (properties) and operations (methods) and the relationships between them.

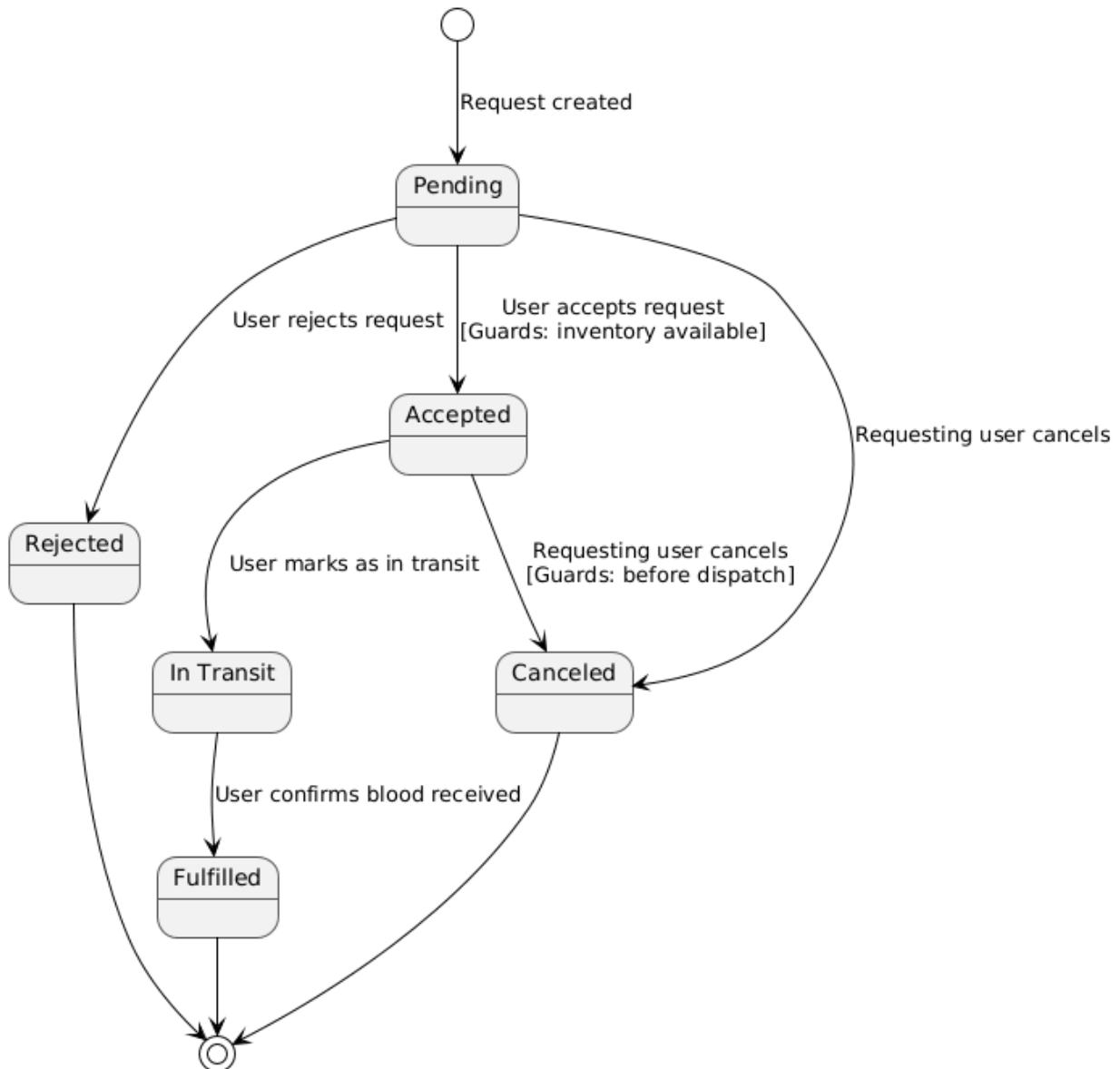


3.2 Sequence Diagram

Please use 'loption handwritten true' to enable handwritten



3.2 State chart Diagram (optional element)



4. Detailed Design

This section provides a detailed specification for each class identified in the Object Model (Section 3.1). The descriptions include class attributes with their data types, visibility, and invariants (validation rules), as well as class operations with their visibility, return types, arguments, and pre/post-conditions.

4.1 The Facility Class

The Facility class represents a physical hospital, clinic, or health center that participates in the Virtual Blood Bank network.

Table 1: Facility Class

Facility
+ facilityId: UUID + name: String + district: String + contactPhone: String + isActive: Boolean + address: Address
+ updateInventory(bloodType, units, expiry): BloodInventory + getInventorySummary(): Map<BloodType, int> + addStaff(user: User): boolean

Table 2: Attributes Description for Facility Class

Attribute	Type	Visibility	Invariant
facilityId	UUID	Public	facilityId <> NULL. Must be a system-generated, unique identifier.
name	String	Public	name <> NULL. Length must be between 3 and 255 characters.

district	String	Public	district <> NULL. Length must be between 3 and 100 characters.
contactPhone	String	Public	contactPhone <> NULL. Must be a valid Ethiopian phone number format (e.g., starts with "+251" or "09" and is 10-13 digits long).
isActive	boolean	Public	isActive <> NULL. Must be either true or false. Default is true.
address	Address	Public	address <> NULL. Must be a valid Address object.

Table 3: Operation Description for Facility Class

Operation	Visibility	Return Type	Argument(s)	Pre-Condition	Post-Condition
updateInventory	Public	BloodInventory	bloodType: BloodType units: int expiry: Date	bloodType must be a valid enum member. units must be a non-negative integer. expiry must be a future date.	A BloodInventory record for the facility is created or updated.

getInventorySummary	Public	Map<BloodType, int>	-	The facility must exist.	A map is returned containing the total units for each blood type in the facility's inventory.
addStaff	Public	boolean	user: User	The user object must be valid and not already associated with another facility.	The user is successfully associated with this facility. Returns true on success, false on failure.

4.2 The Address Class

The Address class is a value object that provides structured location information for a Facility or DonationCenter.

Table 4: Address Class

Address
+ street: String
+ city: String
+ location: GeoPoint

Table 5: Attributes Description for Address Class

Attribute	Type	Visibility	Invariant
street	String	Public	street length must be between 5 and 255 characters. Can be NULL if not applicable.
city	String	Public	city <> NULL. Length must be between 3 and 100 characters.
location	GeoPoint	Public	location <> NULL. Must be a valid geographical coordinate (contains latitude and longitude).

Note: The Address class does not have operations as it is primarily a data structure.

4.3 The User Class

The User class represents a general healthcare professional who can log into and use the system's core features. It serves as the base class for more specialized user roles.

Table 6: User Class

User
+ userId: UUID + fullName: String + phoneNumber: String - email: String - passwordHash: String + role: String + isActive: Boolean - dateCreated: DateTime - lastLogin: DateTime

```
+ login(password): Session
+ logout()
+ changePassword(currentPass, newPass): Boolean
+ viewProfile(): Map<String, String>
```

Table 7: Attributes Description for User Class

Attribute	Type	Visibility	Invariant
userId	UUID	Public	userId <> NULL. Must be a system-generated, unique identifier.
fullName	String	Public	fullName <> NULL. Must contain at least two names and be between 3 and 100 characters. Must not contain special characters or numbers.
phoneNumber	String	Public	phoneNumber <> NULL and must be unique. Must be a valid Ethiopian phone number format (e.g., starts with "+251" or "09" and is 10-13 digits long).
email	String	Private	email can be NULL. If not NULL, it must be unique and follow standard email format rules: must contain "@" and "." symbols, with "." appearing after "@".
passwordHash	String	Private	passwordHash <> NULL. Must be a securely hashed and salted representation of the user's password, not plain text.
role	String	Public	role <> NULL. Must be a value from a predefined list (e.g., "Doctor", "Nurse", "Administrator").

isActive	boolean	Public	isActive <> NULL. Must be either true or false. Default is true. An inactive user cannot log in.
dateCreated	DateTime	Private	dateCreated <> NULL. Must be a system-generated timestamp.
lastLogin	DateTime	Private	lastLogin can be NULL. Must be a system-generated timestamp updated on each successful login.

Table 8: Operation Description for User Class

Operation	Visibility	Return Type	Argument(s)	Pre-Condition	Post-Condition
login	Public	Session	password: String	The user's isActive status must be true.	If the provided password matches the stored passwordHash, a valid Session object is returned. The lastLogin attribute is updated.
logout	Public	void	-	The user must have an active session.	The user's current session is invalidated.

changePassword	Public	boolean	currentPass: String newPass: String	The user must be authenticated . currentPass must match the existing passwordHas h. newPass must meet complexity requirements .	The user's passwordHas h is updated to reflect the newPass. Returns true on success, false on failure.
viewProfile	Public	Map<String, String>	-	The user must be authenticated .	A map containing the user's public profile information (fullName, phoneNumber, role) is returned.

4.4 The AdminUser Class

The AdminUser class inherits from the User class and has additional privileges for managing other users and system settings.

Table 9: AdminUser Class

AdminUser (inherits from User)
+ createUser(details): User + deactivateUser(user: User): Boolean + manageFacility(facility: Facility): boolean

Table 10: Operation Description for AdminUser Class

Operation	Visibility	Return Type	Argument(s)	Pre-Condition	Post-Condition
createUser	Public	User	details: Map<String, String>	The current user's role must be "Administrator". The details map must contain all required fields for a new User.	A new User object is created and persisted in the system. The new User object is returned.
deactivateUser	Public	boolean	user: User	The current user's role must be "Administrator". The target user must exist and not be the administrator themselves.	The target user's isActive attribute is set to false. Returns true on success.
manageFacility	Public	boolean	facility: Facility	The current user's role must be "Administrator".	The details of the facility (e.g., name, contact info) are updated in the system. Returns true on success.

4.5 The BloodInventory Class

The BloodInventory class represents the stock of a single blood type within a specific facility. It is responsible for tracking the quantity and expiry of blood units.

Table 11: BloodInventory Class

BloodInventory
+ inventoryId: UUID + bloodType: BloodType - unitsAvailable: int - expiryDate: Date - lastUpdated: DateTime
+ addUnits(count, expiry): boolean + removeUnits(count): boolean + isExpired(): Boolean + getUnitsAvailable(): int

Table 12: Attributes Description for BloodInventory Class

Attribute	Type	Visibility	Invariant
inventoryId	UUID	Public	inventoryId <> NULL. Must be a system-generated, unique identifier.
bloodType	BloodType	Public	bloodType <> NULL. Must be a valid member of the BloodType enumeration.
unitsAvailable	int	Private	unitsAvailable <> NULL. Must be a non-

			negative integer (≥ 0).
expiryDate	Date	Private	expiryDate $\neq$ NULL. Must be a valid date. Can be in the past (to indicate expired stock).
lastUpdated	DateTime	Private	lastUpdated $\neq$ NULL. Must be a system-generated timestamp, updated on any change to unitsAvailable.

Table 13: Operation Description for BloodInventory Class

Operation	Visibility	Return Type	Argument(s)	Pre-Condition	Post-Condition
addUnits	Public	boolean	count: int expiry: Date	count must be a positive integer. expiry must be a future date.	unitsAvailable is incremented by count. The lastUpdated timestamp is updated. Returns true on success.
removeUnits	Public	boolean	count: int	count must be a positive integer. The current unitsAvailable must be $\geq$ count.	unitsAvailable is decremented by count. The lastUpdated timestamp is updated. Returns true on success, false otherwise.

4.6 The BloodRequest Class

Table 14: BloodRequest Class

32 | Page


```

+ acceptRequest(): boolean
+ rejectRequest(reason): boolean
+ markAsInTransit(): Boolean
+ fulfillRequest(): Boolean
+ cancelRequest(): boolean
    
```

Table 15: Attributes Description for BloodRequest Class

Attribute	Type	Visibility	Invariant
requestId	UUID	Public	requestId <> NULL. Must be a system-generated, unique identifier.
requestedBloodType	BloodType	Public	requestedBloodType <> NULL. Must be a valid member of the BloodType enumeration.
unitsRequested	int	Public	unitsRequested <> NULL. Must be a positive integer (> 0).
status	RequestStatus	Public	status <> NULL. Must be a valid member of the RequestStatus enumeration. Initial state is PENDING.
requestTimestamp	DateTime	Private	requestTimestamp <> NULL. Must be a system-generated timestamp upon creation.

lastUpdateTimestamp	DateTime	Private	lastUpdateTimestamp <> NULL. Must be updated whenever the status changes.
notes	String	Private	notes can be NULL. If not NULL, length must be <= 500 characters.

Table 16: Operation Description for BloodRequest Class

Operation	Visibility	Return Type	Argument(s)	Pre-Condition	Post-Condition
acceptRequest	Public	boolean	-	status must be PENDING. The supplying facility must have sufficient inventory.	status is changed to ACCEPTED. lastUpdateTimestamp is updated. A notification is triggered. Returns true.
rejectRequest	Public	boolean	reason: String (optional)	status must be PENDING.	status is changed to REJECTED. lastUpdateTimestamp is updated. A notification is triggered. Returns true.
markAsInTransit	Public	boolean	-	status must be ACCEPTED.	status is changed to IN_TRANSIT. lastUpdateTimestamp is updated. A notification is triggered. Returns

					true.
fulfillRequest	Public	boolean	-	status must be IN_TRANSIT.	status is changed to FULFILLED. lastUpdateTime is updated. A notification is triggered. Returns true.
cancelRequest	Public	boolean	-	status must be PENDING or ACCEPTED.	status is changed to CANCELED. lastUpdateTime is updated. A notification is triggered. Returns true.

4.7 The Session Class

The Session class represents an authenticated user's session. It is created upon successful login and is used to authorize subsequent user actions.

Table 17: Session Class

Session
- sessionId: UUID + authToken: String - creationTime: DateTime

- expiryTime: DateTime
+ isValid(): boolean
+ refresh(): boolean

Table 18: Attributes Description for Session Class

Attribute	Type	Visibility	Invariant
sessionId	UUID	Private	sessionId <> NULL. Must be a system-generated, unique identifier.
authToken	String	Public	authToken <> NULL. Must be a system-generated, cryptographically secure, unique token string. This is sent to the client.
creationTime	DateTime	Private	creationTime <> NULL. System-generated timestamp when the session is created.
expiryTime	DateTime	Private	expiryTime <> NULL. Timestamp indicating when the session will become invalid (e.g., creationTime + 30 minutes).

Table 19: Operation Description for Session Class

Operation	Visibility	Return Type	Argument(s)	Pre-Condition	Post-Condition
isValid	Public	boolean	-	The session object must	Returns true if the current time is before the expiryTime, false

				exist.	otherwise.
refresh	Public	boolean	-	The session must be valid.	The expiryTime is extended (e.g., by another 30 minutes). Returns true.

4.8 The Notification Class

The Notification class represents an alert or message sent to a user in response to a system event.

Table 20: Notification Class

Notification
+ notificationId: UUID + message: String + type: NotificationType - timestamp: DateTime + isRead: boolean
+ markAsRead(): void

Table 21: Attributes Description for Notification Class

Attribute	Type	Visibility	Invariant
notificationId	UUID	Public	notificationId <> NULL. Must be a system-generated, unique identifier.
message	String	Public	message <> NULL. Length must be between 10 and 500 characters.
type	NotificationType	Public	type <> NULL. Must be a valid member of the NotificationType enumeration.
timestamp	DateTime	Private	timestamp <> NULL. System-generated timestamp upon creation.
isRead	boolean	Public	isRead <> NULL. Must be either true or false. Default is false.

Table 22: Operation Description for Notification Class

Operation	Visibility	Return Type	Argument(s)	Pre-Condition	Post-Condition
markAsRead	Public	void	-	The notification object must exist. isRead must be false.	The isRead attribute is set to true.

4.9 The DonationCenter Class

The DonationCenter class stores informational data about physical locations where blood donations can be made.

Table 23: DonationCenter Class

DonationCenter
+ centerId: UUID + name: String + address: Address + operatingHours: String + contactInfo: String

Table 24: Attributes Description for DonationCenter Class

Attribute	Type	Visibility	Invariant
centerId	UUID	Public	centerId <> NULL. Must be a system-generated, unique identifier.
name	String	Public	name <> NULL. Length must be between 3 and 255 characters.
address	Address	Public	address <> NULL. Must be a valid Address object.
operatingHours	String	Public	operatingHours can be NULL. If not NULL, length must be <= 255 characters (e.g., "Mon-Fri 9AM-5PM").
contactInfo	String	Public	contactInfo can be NULL. If not NULL, must be valid contact information (e.t., a phone number

			or email address).
--	--	--	--------------------

References

- ISO/IEC/IEEE. (2011). *Systems and software engineering — Recommended practice for architectural description (ISO/IEC/IEEE 42010:2011)*.
<https://www.iso.org/standard/50508.html>
- Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2nd ed.). Addison-Wesley.
- Fowler, M. (2003). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed.). Addison-Wesley.
- Sommerville, I. (2011). *Software Engineering* (9th ed.). Pearson Higher Ed.
- Pressman, R. S., & Maxim, B. (2014). *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw-Hill.
- IBM. (n.d.). *UML basics: The Unified Modeling Language*. IBM Developer.
<https://developer.ibm.com/articles/uml-basics-intro/>
- GeeksforGeeks. (2022). *Software Design Document – Format & Example*.
<https://www.geeksforgeeks.org/software-design-document-format/>
- Lucidchart. (n.d.). *UML Diagram Types Guide*.
<https://www.lucidchart.com/pages/uml-diagram>
- IEEE. (1998). *IEEE Std 1016-1998 – Recommended Practice for Software Design Descriptions*.
<https://ieeexplore.ieee.org/document/743771>
- Wikipedia contributors. (2025, January 10). *Unified Modeling Language*. Wikipedia.
https://en.wikipedia.org/wiki/Unified_Modeling_Language

